Calling sequence inside a factory takes one argument (which is the attribute whose values are being sequenced) and a block. When the factory is invoked, the block is called with a new sequential value and the block's return value is set to be the value of the attribute. The start value is 1 by default, but a second argument to sequence can be used to specify an arbitrary value, which is usually a number but can be any object that responds to next.

When a sequence is defined inside a factory, it can be used in only that one place; however, sequences can also be defined outside of factories and reused:

```
FactoryBot.define do
  sequence :email do |n|
    "user_#{n}@test.com"
  end

  factory :user do
    name "Fred Flintstone"
    email
  end
end
```

The use of email inside the factory is a shortcut that assumes the sequence and the attribute have the same name. If so, the sequence is triggered and the next value becomes the value of the attributes. If the sequence and the attribute have different names, then you need to invoke the sequence explicitly by calling generate inside an attribute's block:

```
factory :task do
  title "Finish Chapter"
  user_email { generate(:email) }
end
```

Inherited Factories

Often you'll need to create multiple factories from the same class—a classic example is the ability to create different kinds of users, such as regular users versus administrators.

The most direct way to create slightly different factories in the same class is via factory_bot's inheritance feature. If you define a factory as having another factory as a parent, it takes all the attributes set in that parent but then allows you to override those attributes in the child factory. Effectively, factory_bot inheritance allows you to group common attributes so they can be reused:

```
FactoryBot.define do
  factory :task do
    sequence(:title) { |n| "Task #{n}" }
  end
```